

Repaso Parcial 2

Métricas:

Pueden ser métricas de control o de predicción

1. De control o de proceso, apoyan la gestión del proceso.
 1. Esfuerzo promedio
 2. Tiempo requerido para reparar los defectos reportados
2. De predicción o de producto, ayudan a predecir las características del software. Se asocian con el software en sí.
 1. Complejidad ciclomática, mide los distintos caminos de ejecución que puede tener un código o software completo. (Distintos caminos que puede tomar por ejemplo si es mayor de edad o no)
 - 2.

En el software hay métricas para:

- El producto (el software terminado)
- El proceso
- El proyecto (es un plan de trabajo), presupuesto

Indicador, es una métrica o combinación de métricas que proporcionan comprensión acerca del proceso del software o el producto en sí.

- Índice Fog, mide la facilidad de lectura de un texto

Medición:

Ayuda a la mejora continua, porque permite comparar con algo y así no repetir errores.

Ayuda a determinar tiempos, costos, cantidad de personas y la calidad del mismo.

se ocupa de derivar un valor numérico o perfil para un atributo de un componente, sistema o proceso de software. Al comparar dichos valores unos con otros, y con los estándares que se aplican a través de una organización, es posible extraer conclusiones sobre la calidad del software, o valorar la efectividad de los procesos, las herramientas y los métodos de software.

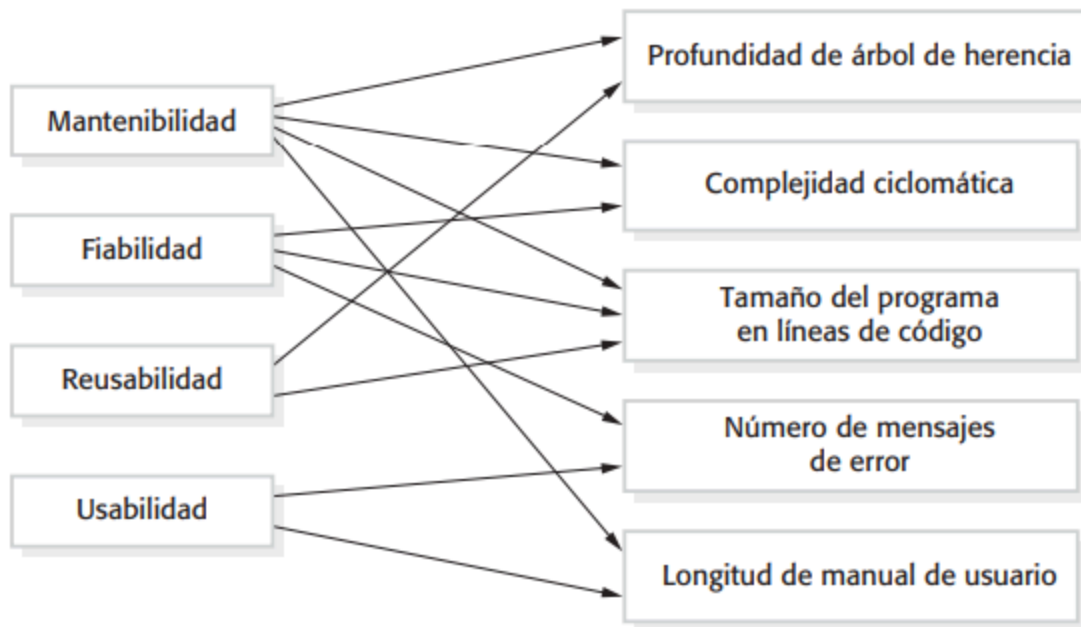
Uso de la medición en un software:

- Para asignar un valor a los distintos atributos de calidad del sistema

- Para identificar los componentes del sistema cuya calidad está por debajo de un estándar

Atributos de calidad externos

Atributos internos



Métricas de producto

Se dividen en dos tipos:

1. Métricas dinámicas, se llevan a cabo mientras el software están en ejecución, útiles para evaluar eficiencia y fiabilidad (ejemplo: número de fallos).
2. Métricas estáticas, el software no está en ejecución, derivadas del código y documentación, útiles para valorar complejidad y mantenibilidad (ejemplo: tamaño del código).

Métricas más populares:

- Fan-in: miden cuántos módulos llaman (dependen de) un módulo en particular.
 - Un fan-in alto puede decir que el módulo es muy reutilizado, si está bien diseñado puede ser positivo.
- Fan-out: mide cuántos módulos son llamados (dependen de) por un módulo en particular.
 - Un fan-out puede ser una señal de acoplamiento excesivo, lo que puede hacer complicado el mantenimiento.
 - Se recomienda un fan-in alto y un fan-out bajo, para mejorar la reutilización y reducir el acoplamiento.
- Longitud de código
- Complejidad ciclomática: mide los distintos caminos de ejecución que puede tener un código o software completo. (Distintos caminos que puede tomar por ejemplo si es mayor de edad o no)

- Longitud de identificadores: medida de la longitud de las variables, si el nombre de las variables es más largo pueden ser más significativas y por lo tanto más entendibles
- Índice fog, mide la facilidad de lectura de un texto

Métricas orientadas a objetos

Algunas métricas clave en software orientado a objetos incluyen:

- **Métodos ponderados por clase (WMC):** mide la cantidad de métodos en una clase, ponderados por su complejidad. Valores altos sugieren clases más complejas y difíciles de reutilizar.
- **Profundidad de árbol de herencia (DIT):** indica la cantidad de niveles en la jerarquía de herencia. Un DIT alto puede hacer que el diseño sea más difícil de comprender.
- **Número de hijos (NOC):** representa la cantidad de subclases directas de una clase. Un valor alto puede indicar mayor reutilización, pero también mayor esfuerzo en validación.
- **Acoplamiento entre clases de objetos (CBO):** evalúa la dependencia entre clases. Un CBO elevado sugiere que modificaciones en una clase pueden afectar a otras.
- **Respuesta por clase (RFC):** número de métodos que pueden ejecutarse en respuesta a un mensaje recibido. Valores altos sugieren mayor complejidad y probabilidad de errores.
- **Falta de cohesión en métodos (LCOM):** mide la cohesión dentro de una clase, comparando métodos que comparten o no atributos. Existen múltiples variaciones de esta métrica, pero su utilidad sigue en debate.

Análisis de componentes de software

El análisis de componentes del software implica medir diferentes aspectos de los módulos y compararlos con datos históricos. Mediciones anómalas pueden indicar problemas de calidad en ciertos componentes.

Las etapas clave en este proceso incluyen:

1. **Elegir las mediciones a realizar:** se formulan preguntas clave y se define qué métricas recopilar.
2. **Seleccionar componentes a valorar:** no siempre es necesario medir todos los módulos, sino que puede seleccionarse una muestra representativa.
3. **Medir características de los componentes:** se usan herramientas automatizadas para calcular valores métricos.
4. **Identificar mediciones anómalas:** se comparan valores con datos históricos y se identifican valores inusuales.
5. **Analizar componentes anómalos:** se evalúan los módulos con valores anormales para determinar si reflejan problemas de calidad reales.

Se recomienda almacenar datos históricos de métricas para mejorar la evaluación de calidad del software en futuros proyectos y validar relaciones entre métricas internas y calidad externa.

Ambigüedad de mediciones

Las mediciones de software deben analizarse en contexto, ya que pueden ser malinterpretadas. Un ejemplo es la relación entre peticiones de cambio y la calidad del software:

- Se podría asumir que un alto número de peticiones de cambio indica baja calidad.
- Sin embargo, también podría significar que el software es popular y ampliamente utilizado.
- Cambios en los procesos pueden influir en la cantidad de peticiones sin necesariamente reflejar mejoras o deterioros en la calidad.

Para interpretar correctamente los datos, es necesario conocer quién realiza las peticiones, su motivo y el contexto del mercado. Los datos cuantitativos por sí solos no siempre reflejan la realidad y deben analizarse con precaución antes de extraer conclusiones definitivas.